

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — ANNEXURE A

DESIGN TASK

Course: Artificial Intelligence (CSA17)

Course Outcome: CO5 — Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

Note: All five design questions from Annexure A are answered below, each addressing every rubric criterion — problem understanding, system architecture, PySpark/RDD usage, intelligent agent integration, scalability, data flow, and innovation.

Design 1: Hybrid AI Recommendation Engine

1. Problem Understanding & Requirement Analysis

Problem Statement: E-commerce and content platforms need a recommendation system that gives relevant, personalized suggestions at scale, updates in real time as user behavior changes, and does not suffer from either the cold-start problem (new items with no ratings) or the popularity-bias problem (over-recommending only popular items) that pure collaborative or pure content-based systems face individually.

Objectives:

- Combine content-based filtering (item feature similarity) and collaborative filtering (user-item interaction patterns) into a single hybrid engine.
- Process large-scale, continuously arriving user-item interaction data using distributed computing.
- Adapt recommendations in real time as new clicks, ratings, and feedback arrive.

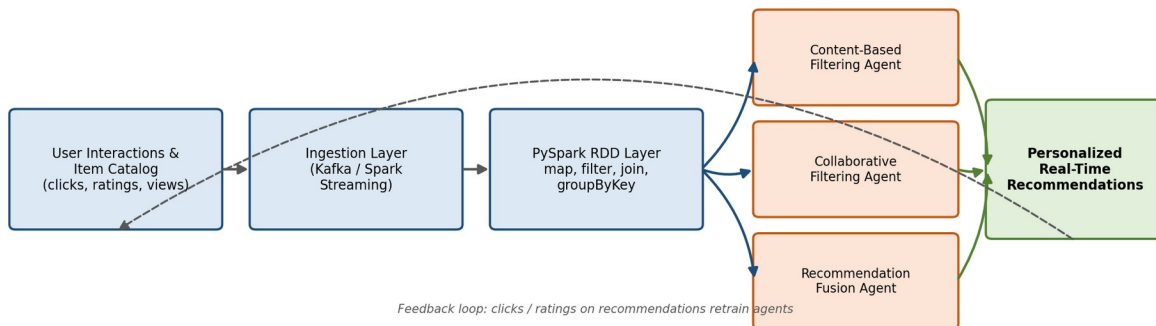
Constraints:

- Must handle millions of users and items without a single-machine bottleneck.
- Cold-start items/users must still receive reasonable recommendations.
- Recommendation latency must stay low enough for a live user-facing product.

2. System Architecture Design

The system is organized in four layers. The ingestion layer captures streaming user events (clicks, ratings, views) and the static item catalog via Kafka feeding into Spark Streaming. The processing layer uses PySpark RDDs to clean, join, and aggregate this data into two feature sets: item-feature vectors (for content-based filtering) and user-item interaction matrices (for collaborative filtering). The intelligent agent layer contains a Content-Based Filtering Agent, a Collaborative Filtering Agent, and a Recommendation Fusion Agent that weights and merges the two agents' outputs based on data availability (e.g., leaning on content-based scores for cold-start items). The output layer serves personalized ranked recommendations back to the application and logs the user's response as new training feedback.

Hybrid AI Recommendation Engine — Architecture & Data Flow



3. Data Flow & Processing Pipeline

- Stage 1 — Raw interaction events and catalog updates land in the ingestion layer via Spark Streaming micro-batches.
- Stage 2 — PySpark RDDs clean and transform raw events into structured (user_id, item_id, rating/implicit-signal) tuples.
- Stage 3 — Content and interaction RDDs are partitioned and cached for repeated agent access.
- Stage 4 — Agents independently score candidate items; the Fusion Agent combines both scores into a final ranked list.
- Stage 5 — Recommendations are pushed to the user interface; resulting clicks/ratings are captured as new events, closing the feedback loop.

4. Use of PySpark & RDD Operations

PySpark RDDs provide the distributed backbone that lets this pipeline scale to large interaction logs without loading everything into a single machine's memory.

RDD Operation	Role in This System
map	Transforms raw JSON/log events into structured (user, item, rating) tuples.
filter	Removes bot traffic, duplicate events, and malformed records before feature extraction.
groupByKey / reduceByKey	Aggregates all interactions per user or per item to build interaction profiles and item popularity counts.
join	Merges user-interaction RDDs with the item-catalog RDD to attach item metadata (category, tags, price) needed for content-based scoring.
flatMap	Expands multi-valued item tags/categories into individual key-value records for content similarity computation.

5. Intelligent Agent Integration

- Content-Based Filtering Agent: computes similarity between a user's historically liked item features and candidate items, handling cold-start items well since it needs no interaction history for the new item.
- Collaborative Filtering Agent: identifies users/items with similar interaction patterns (e.g., via matrix factorization on the RDD-built interaction matrix) to surface items liked by similar users.
- Recommendation Fusion Agent: dynamically weights the two agents' outputs — favoring content-based scores when interaction data is sparse and collaborative scores when it is rich — and returns one ranked list.

The three agents operate asynchronously on shared PySpark-processed RDDs and communicate only through the scores they emit; the Fusion Agent is the sole coordination point, which keeps the design loosely coupled and lets each filtering agent be retrained or replaced independently.

6. Scalability & Distributed Computing Design

- RDD partitioning by `user_id` / `item_id` allows agent computation to be distributed across the cluster with data locality.
- Caching (`persist()`) of frequently reused interaction and feature RDDs avoids recomputation across pipeline stages.
- Spark's lineage-based fault tolerance means a lost partition is recomputed automatically from source RDDs rather than requiring full pipeline restarts.
- Horizontal scaling: adding worker nodes increases throughput roughly linearly since user/item partitions are largely independent.

7. Innovation & Creativity

The novelty is in the adaptive fusion strategy: rather than using a fixed weighted average of the two filtering techniques, the Fusion Agent adjusts its blend ratio per user/item based on how much interaction data is available, directly targeting the cold-start weakness of collaborative filtering while retaining its stronger personalization once enough data accumulates.

Design 2: Smart Disaster Detection System

1. Problem Understanding & Requirement Analysis

Problem Statement: Natural disasters such as floods and earthquakes require early detection from heterogeneous, high-volume data sources (satellite imagery, IoT ground sensors, and social media reports) so that warnings and response coordination can happen before damage escalates, but no single data source is reliable or fast enough alone.

Objectives:

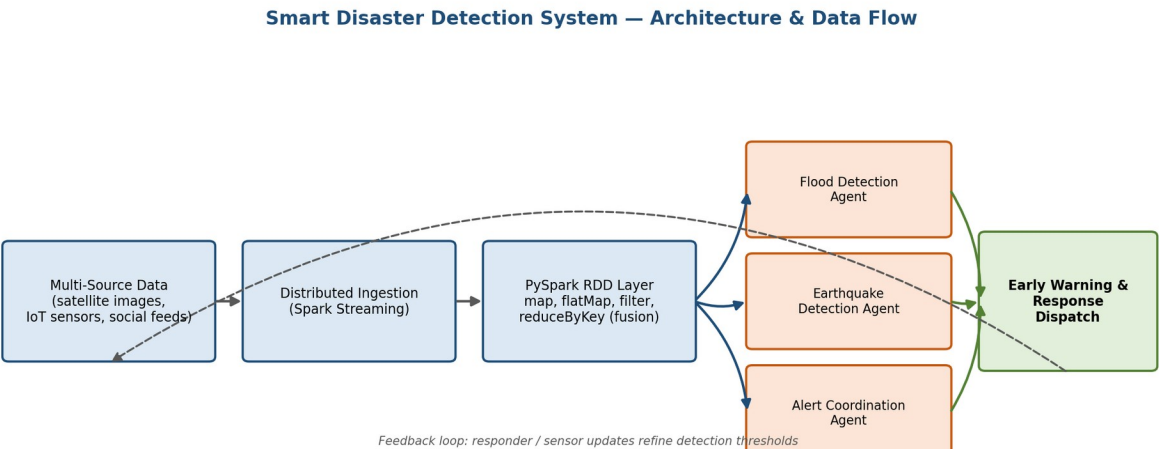
- Fuse satellite imagery, sensor telemetry, and social media signals into a unified, real-time disaster-risk picture.
- Detect flood and earthquake events with minimal latency using distributed processing.
- Coordinate automated early-warning alerts and suggested response actions.

Constraints:

- Data arrives continuously and at very high volume (especially imagery and sensor streams), demanding streaming rather than batch-only processing.
- Social media text is noisy and unstructured, requiring robust cleaning before use.
- False alarms carry real cost, so fusion logic must cross-validate across sources before triggering alerts.

2. System Architecture Design

Data enters through a distributed ingestion layer built on Spark Streaming, pulling from satellite image feeds, sensor telemetry APIs, and social media streams simultaneously. PySpark RDDs clean and normalize each source into a common schema, then a Data Fusion Agent merges spatially and temporally aligned records. Two specialized detection agents — a Flood Detection Agent and an Earthquake Detection Agent — independently score fused records for their respective hazard types, and an Alert Coordination Agent cross-checks agent outputs against each other and against a confidence threshold before dispatching an early warning.



3. Data Flow & Processing Pipeline

- Stage 1 — Satellite images, sensor readings, and social posts are ingested concurrently via Spark Streaming.

- Stage 2 — PySpark RDDs apply map/filter to standardize formats and discard noise (e.g., irrelevant social posts).
- Stage 3 — flatMap parses free-text social posts into keyword/location tokens; reduceByKey aggregates signals by geographic region and time window.
- Stage 4 — The Data Fusion Agent joins region-aligned records across all three sources into a single per-region risk record.
- Stage 5 — Detection agents score each fused record; the Alert Coordination Agent issues warnings only when multiple sources agree above threshold.

4. Use of PySpark & RDD Operations

Because disaster data is both high-volume and multi-format, PySpark RDD operations are used to normalize and aggregate each source before any detection logic runs.

RDD Operation	Role in This System
map	Converts raw sensor readings and image metadata into a common (region, timestamp, signal_type, value) schema.
filter	Discards corrupted sensor readings, duplicate social posts, and out-of-region data.
flatMap	Breaks social media text into individual keyword/hashtag tokens for disaster-relevant term detection.
reduceByKey	Aggregates signal counts and intensities per geographic grid cell and time window across all three sources.
join	Aligns sensor, imagery, and social-signal RDDs on the same (region, time-window) key to build fused risk records.

5. Intelligent Agent Integration

- Flood Detection Agent: applies rainfall/water-level thresholds and image-based water coverage estimates to the fused record to flag flood risk.
- Earthquake Detection Agent: analyzes seismic sensor patterns and sudden spikes in location-tagged social reports to flag seismic events.
- Alert Coordination Agent: reconciles the two detection agents' outputs, resolves conflicting signals, and decides whether the combined confidence justifies issuing a public warning.

Detection agents run independently and in parallel on the same fused RDD partitions (one per hazard type), and only the Alert Coordination Agent synthesizes their outputs — this keeps hazard-specific logic modular so new hazard-detection agents (e.g., wildfire) can be added without touching existing agents.

6. Scalability & Distributed Computing Design

- Streaming micro-batches are partitioned by geographic region, allowing regional detection to scale horizontally with added worker nodes.
- Checkpointing in Spark Streaming ensures the pipeline can recover its state after a worker failure without losing in-flight disaster data.
- Windowed aggregation (reduceByKey over sliding time windows) bounds memory use even as social/sensor volume spikes during an actual event.
- Fault tolerance via RDD lineage means a failed detection task is recomputed from the last fused RDD checkpoint, not from raw ingestion.

7. Innovation & Creativity

The innovation lies in cross-source corroboration before alerting: rather than trusting any single feed (which is prone to sensor faults or social media noise), the Alert Coordination Agent requires agreement across independently computed hazard scores, meaningfully reducing false alarms while keeping detection latency low through parallel agent execution.

Design 3: AI-Based Fraud Detection Framework

1. Problem Understanding & Requirement Analysis

Problem Statement: Financial institutions process massive volumes of transactions per second and need to detect fraudulent patterns (card, online, ATM) in near real time, but fraud patterns evolve constantly and a centralized, non-scalable detection system cannot keep pace with transaction volume or adapt quickly enough to new fraud tactics.

Objectives:

- Process large-scale, high-velocity transaction streams using distributed computing.
- Detect anomalous and suspicious transaction patterns with low latency.
- Continuously adapt detection logic as fraud patterns evolve, minimizing false positives that inconvenience legitimate customers.

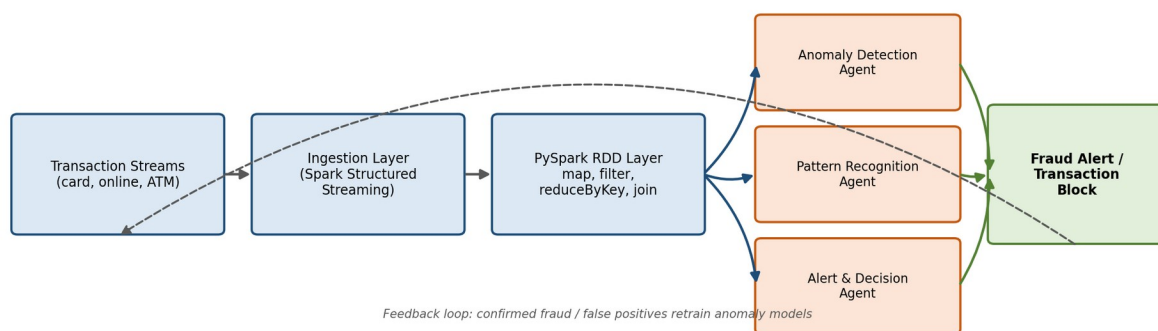
Constraints:

- Detection must complete within the transaction-authorization window (typically sub-second to a few seconds).
- The system must handle strong class imbalance — genuine fraud is a small fraction of all transactions.
- Sensitive financial data requires secure, auditable processing.

2. System Architecture Design

Transaction streams (card, online, ATM) flow into a Spark Structured Streaming ingestion layer. PySpark RDDs clean, enrich, and aggregate transactions, joining each transaction with the customer's recent account history. An Anomaly Detection Agent flags statistical outliers (e.g., unusual amount, location, or frequency), a Pattern Recognition Agent matches transactions against known fraud signatures and sequences, and an Alert & Decision Agent combines both agents' confidence scores to decide whether to allow, flag, or block a transaction in real time.

AI-Based Fraud Detection Framework — Architecture & Data Flow



3. Data Flow & Processing Pipeline

- Stage 1 — Transactions stream in continuously and are ingested in micro-batches via Spark Structured Streaming.
- Stage 2 — PySpark RDDs apply map/filter to standardize transaction records and discard malformed entries.

- Stage 3 — reduceByKey aggregates recent transaction counts/amounts per account to build a rolling behavioral profile.
- Stage 4 — join merges each incoming transaction with the account's rolling profile and historical fraud labels.
- Stage 5 — The Anomaly Detection and Pattern Recognition Agents score the enriched record in parallel; the Alert & Decision Agent issues the final allow/flag/block decision.

4. Use of PySpark & RDD Operations

PySpark RDDs let the framework aggregate rolling account behavior and join it against live transactions fast enough to stay within the authorization window.

RDD Operation	Role in This System
map	Parses raw transaction logs into structured (account_id, amount, location, timestamp, merchant) records.
filter	Removes malformed transactions and pre-filters clearly legitimate low-risk transaction types to reduce downstream load.
reduceByKey	Aggregates transaction count, total amount, and average amount per account over a rolling time window to build a behavioral baseline.
join	Combines the incoming transaction RDD with the account's historical-profile RDD and known fraud-signature RDD.
sortBy / top-N (action)	Ranks flagged transactions by risk score so the highest-risk cases are reviewed or blocked first.

5. Intelligent Agent Integration

- Anomaly Detection Agent: flags transactions that deviate statistically from the account's own historical behavior (unusual amount, location, or timing).
- Pattern Recognition Agent: matches transaction sequences against known fraud signatures and evolving fraud-ring patterns learned from labeled past cases.
- Alert & Decision Agent: fuses both agents' risk scores against a decision policy to allow, flag for review, or automatically block the transaction.

The Anomaly Detection and Pattern Recognition Agents run concurrently on the same enriched transaction RDD partitions, and the Alert & Decision Agent acts as the single arbitration point — this separation lets the statistical (anomaly) and rule/pattern-based detection logic be updated independently as fraud tactics change.

6. Scalability & Distributed Computing Design

- Transaction RDDs are partitioned by account_id, allowing per-account aggregation and joins to be distributed across the cluster.
- Micro-batch streaming windows bound the state that must be held in memory even under transaction-volume spikes.
- Spark's RDD lineage enables automatic recomputation of a lost partition without reprocessing the entire transaction stream.
- New worker nodes can be added to linearly increase throughput as transaction volume grows, without redesigning the pipeline.

7. Innovation & Creativity

The framework's innovation is combining a statistical anomaly detector (which catches novel, previously unseen fraud patterns) with a signature/pattern-matching agent (which catches known fraud-ring behavior) and letting the Alert & Decision Agent weigh both — this dual-agent approach catches both emerging and known fraud types better than either technique alone, while keeping each detection agent independently retrainable as new labeled fraud cases arrive.

Design 4: Autonomous Supply Chain Optimization System

1. Problem Understanding & Requirement Analysis

Problem Statement: Modern supply chains must continuously balance inventory levels, logistics routing, and fluctuating demand across many warehouses and regions; manual or centralized planning cannot react fast enough to disruptions (demand spikes, delivery delays, stockouts), leading to excess cost or lost sales.

Objectives:

- Forecast demand and optimize inventory and logistics decisions using distributed data processing.
- Allow decentralized agents to make local decisions that still converge on a globally efficient plan.
- Minimize total supply chain cost while maintaining service-level targets.

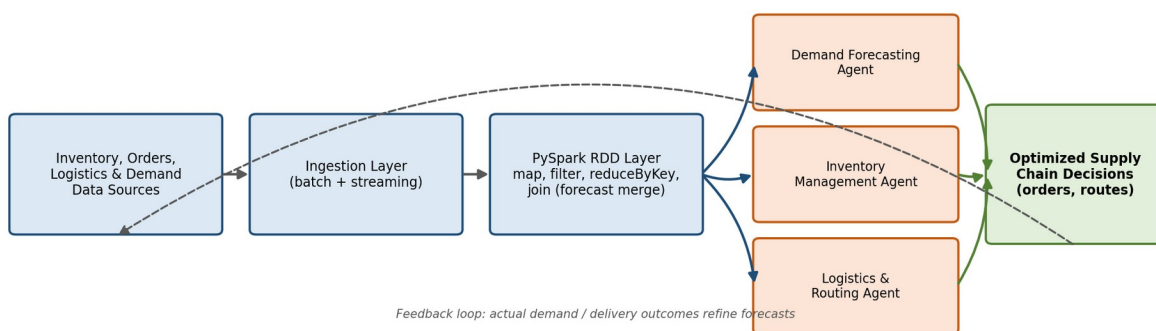
Constraints:

- Data spans many warehouses, suppliers, and transport routes, requiring distributed rather than single-node computation.
- Demand forecasts must update as new order and market data arrives, not just on a fixed daily batch cycle.
- Logistics decisions must respect real-world constraints (vehicle capacity, delivery windows, supplier lead times).

2. System Architecture Design

Inventory levels, order history, logistics data, and external demand signals feed into a batch-plus-streaming ingestion layer. PySpark RDDs clean, join, and aggregate this data into per-product, per-location feature sets used for forecasting. A Demand Forecasting Agent predicts near-term demand per location; an Inventory Management Agent uses those forecasts to recommend reorder quantities and safety stock; a Logistics & Routing Agent optimizes delivery routes and shipment schedules given inventory decisions. The three agents interact iteratively — forecast changes ripple into inventory recommendations, which in turn constrain routing — before final decisions are issued.

Autonomous Supply Chain Optimization — Architecture & Data Flow



3. Data Flow & Processing Pipeline

- Stage 1 — Inventory, order, logistics, and demand-signal data are ingested from multiple systems (ERP, POS, telematics) via batch and streaming pipelines.
- Stage 2 — PySpark RDDs apply map/filter to standardize records per product-location key.

- Stage 3 — reduceByKey aggregates historical order volume and inventory turnover per product-location.
- Stage 4 — join merges the aggregated demand history with current inventory levels and active logistics constraints.
- Stage 5 — The three agents consume the joined RDD, iterate on forecasts, inventory targets, and routes, and emit a final coordinated supply-chain plan.

4. Use of PySpark & RDD Operations

PySpark RDDs aggregate and join transactional data across many warehouses and products so forecasting and optimization agents work from a single consistent, distributed feature set.

RDD Operation	Role in This System
map	Normalizes heterogeneous ERP/POS/telematics records into a common (product, location, timestamp, quantity) schema.
filter	Removes canceled orders, duplicate shipment records, and out-of-range sensor/telematics readings.
reduceByKey	Aggregates historical demand and inventory turnover per (product, location) key to feed the forecasting agent.
join	Merges demand-history RDDs with current inventory-level RDDs and active logistics-constraint RDDs.
groupByKey	Groups shipment records by route/vehicle for the Logistics & Routing Agent's optimization step.

5. Intelligent Agent Integration

- Demand Forecasting Agent: predicts short-term product demand per location from aggregated historical and current order data.
- Inventory Management Agent: translates demand forecasts into reorder quantities and safety-stock levels per warehouse.
- Logistics & Routing Agent: optimizes delivery routes and shipment schedules subject to vehicle capacity and delivery-window constraints, given the inventory agent's decisions.

The three agents operate in a decentralized but sequentially-informed loop — forecasting feeds inventory decisions, which feed routing decisions — with each agent able to trigger a re-evaluation upstream (e.g., a routing constraint infeasibility can prompt the inventory agent to revise a reorder decision), giving the system a degree of autonomous, decentralized negotiation rather than a single rigid top-down plan.

6. Scalability & Distributed Computing Design

- Partitioning RDDs by product-location key allows forecasting and inventory computations to run independently and in parallel across the cluster.
- Incremental/streaming updates let the pipeline react to new orders or disruptions without recomputing the entire historical aggregation.
- RDD lineage-based fault tolerance ensures a failed worker's partition (e.g., one region's data) is recomputed without restarting the whole optimization run.
- The architecture scales horizontally as new warehouses or product lines are added, since each product-location key is processed independently.

7. Innovation & Creativity

The innovation is in making the three agents interact iteratively rather than in a single fixed sequence — inventory and routing feasibility issues can push a re-forecast or re-plan upstream, giving the system a form of decentralized negotiation that adapts to real-world constraints instead of relying on one-shot, top-down optimization.

Design 5: Personalized Learning Analytics Platform

1. Problem Understanding & Requirement Analysis

Problem Statement: Online learning platforms serving thousands of learners generate huge volumes of behavioral and performance data (LMS logs, quiz scores, engagement metrics), but most platforms still deliver the same static content to everyone, missing the opportunity to personalize pacing and content to each learner's actual needs at scale.

Objectives:

- Analyze learner behavior and performance data at scale using distributed processing.
- Build individual learner profiles that capture strengths, weaknesses, and engagement patterns.
- Adaptively personalize content delivery and learning paths in near real time for thousands of concurrent learners.

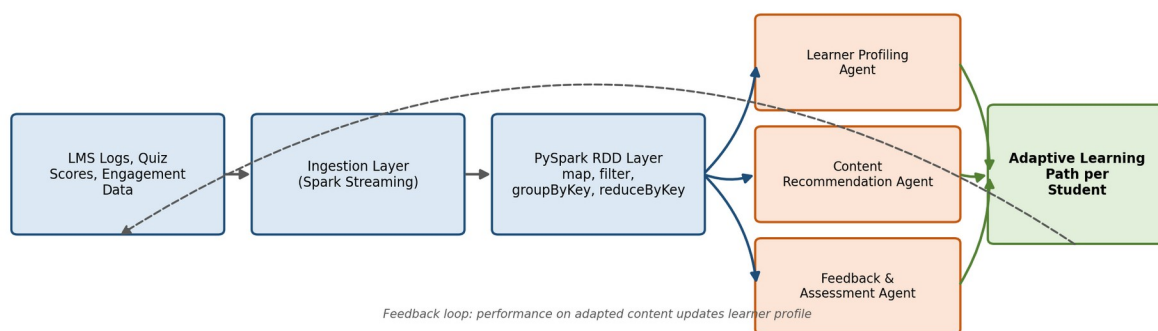
Constraints:

- Data volume grows continuously as learners interact with the platform, requiring streaming-capable processing.
- Personalization must update quickly enough to affect the learner's current session, not just future ones.
- The system must scale to thousands of simultaneous learners without degrading responsiveness.

2. System Architecture Design

LMS interaction logs, quiz scores, and engagement metrics stream into a Spark Streaming ingestion layer. PySpark RDDs clean and aggregate this data per learner. A Learner Profiling Agent builds and updates each learner's skill/engagement profile; a Content Recommendation Agent selects the next best content or exercise for that learner; a Feedback & Assessment Agent evaluates the learner's response to adapted content and reports outcomes back into the profiling agent, closing the loop.

Personalized Learning Analytics Platform — Architecture & Data Flow



3. Data Flow & Processing Pipeline

- Stage 1 — LMS logs, quiz results, and engagement events are ingested continuously via Spark Streaming.
- Stage 2 — PySpark RDDs apply map/filter to standardize and clean events per learner.

- Stage 3 — groupByKey groups all events for a given learner into a single processing unit.
- Stage 4 — reduceByKey aggregates quiz scores and time-on-task metrics per learner per topic.
- Stage 5 — The Learner Profiling, Content Recommendation, and Feedback & Assessment Agents consume this aggregated data to produce and continuously refine an adaptive learning path.

4. Use of PySpark & RDD Operations

PySpark RDDs aggregate high-volume, per-learner event streams into structured profiles that the agent layer can act on without waiting for a full batch cycle.

RDD Operation	Role in This System
map	Converts raw LMS/quiz/engagement log entries into structured (learner_id, activity, score, timestamp) records.
filter	Removes incomplete sessions, duplicate submissions, and non-instructional events (e.g., idle pings).
groupByKey	Groups all activity records for a given learner together for profile computation.
reduceByKey	Aggregates quiz scores, attempt counts, and time-on-task per (learner, topic) key.
join	Merges the learner's aggregated performance RDD with the content-catalog RDD (difficulty, topic tags) to match learners to appropriate content.

5. Intelligent Agent Integration

- Learner Profiling Agent: continuously updates each learner's skill-mastery and engagement profile from aggregated activity data.
- Content Recommendation Agent: selects the next best content, exercise, or hint level tailored to the learner's current profile.
- Feedback & Assessment Agent: evaluates how the learner performs on the recommended content and feeds that outcome back into the profile.

The three agents form a continuous closed loop per learner — profiling informs recommendation, recommendation outcomes are assessed, and assessment results update the profile again — allowing the system to adapt within a single learning session rather than only between sessions.

6. Scalability & Distributed Computing Design

- Partitioning learner activity RDDs by learner_id lets profile updates for different learners run fully in parallel across the cluster.
- Streaming micro-batches keep per-update latency low even as thousands of learners generate events concurrently.
- RDD lineage-based fault tolerance recovers a failed partition (e.g., one learner cohort's data) without disrupting other learners' sessions.
- The architecture scales horizontally — adding worker nodes increases the number of learners that can be profiled and served concurrently.

7. Innovation & Creativity

The innovation is closing the profiling–recommendation–assessment loop within the same session rather than only updating profiles overnight in batch, so a learner who struggles with a concept can receive adapted content and be re-assessed before the session ends, rather than waiting until their next login.